

**UNIVERSITY OF THE ARMED FORCES-ESPE
SANTO DOMINGO DE LOS TSÁCHILAS CAMPUS**

DEPARTMENT OF COMPUTER SCIENCE - DCCO-SS

INFORMATION TECHNOLOGY ENGINEERING PROGRAM



PERIOD : 202650 April – August 2026

SUBJECT : Web Systems Fundamentals.

TOPIC : Task: Analysis of Asynchronous Processes

STUDENT : Manzanillas Delgado Klay Shande

LEVEL-PARALLEL – NRC : Fourth - Nrc – 29535

TEACHER : Eng. Geovanny Brito.

SUBMISSION DATE : July 22, 2026.

SANTO DOMINGO – ECUADOR

Content

1. GitHub Link:	4
2. Introduction	4
3. Objectives	4
2.1. General Objective	4
2.2. Specific Objectives:	4
4. Development	5
3.1. Application Structure	5
3.2. Automatic Profile Picture Loading (Dog CEO API)	6
3.3. Loading and Displaying Users from JSONPlaceholder	7
3.4. Adding Local Users and Persistence with localStorage	9
3.5. Storage Comparison: localStorage vs sessionStorage vs Cookies	11
3.6. Loading and Displaying Products from DummyJSON	12
3.7. Basic Store Catalog Simulation	13
3.8. Displaying Cat Facts	13
3.9. Displaying JSON in a Modal with JSON.stringify	14
3.10. Using async/await and the Asynchronous Flow	16
3.11. Error Handling with try...catch	17
3.12. Comparison: Vanilla JavaScript vs jQuery	17
5. Conclusions	18
6. References	18

Code

Code 1 DOMContentLoaded event listener	6
Code 2 loadProfilePicture() function	6
Code 3 loadUsers() function	7
Code 4 renderUsersTable() function	8
Code 5 loadLocalUsersFromStorage() function	9
Code 6 saveLocalUsersToStorage() function	10
Code 7 handleAddUserForm() function	10
Code 8 handleClearLocalUsers() function	10
Code 9 loadProducts() function	12
Code 10 renderProducts() function	12
Code 11 loadCatFact() function	14
Code 12 showJsonModal() function	14
Code 13 Complete async/await flow with error handling	16

Code 14 Error handling pattern	17
--------------------------------------	----

Figure

Figure 1 shows the application homepage with all navigation tabs.....	5
Figure 2 shows the Profile tab with the loaded image.	7
Figure 3 shows the table with API and local users.	9
Figure 4 shows the add user form with completed fields.	10
Figure 5 shows the Research tab with comparative tables	11
Figure 6 shows the product catalog.	13
Figure 7 shows a cat fact displayed in the interface.....	14
Figure 8 JSON Modal showing the users data	15
Figure 9 JSON Modal showing the products data	15
Figure 10 shows the Research tab with comparative tables	18

1. GitHub Link:

<https://github.com/klayshande/mini-store.git>

2. Introduction

This report presents the development of a Mini Store web application created using HTML5, Bootstrap 5, and Vanilla JavaScript. The main purpose of this project is to strengthen essential frontend development skills, including the integration of external APIs, processing JSON data, asynchronous programming with async/await, dynamic DOM manipulation, and client-side data persistence using localStorage.

The application consumes data from four public APIs: JSONPlaceholder for sample user information, DummyJSON for product data, Dog CEO API to display random profile images, and Cat Facts API to provide interesting cat facts. In addition, users can create and save new local user profiles, which are stored in the browser using localStorage, allowing the information to remain available even after the page is refreshed.

3. Objectives

2.1. General Objective

The objective of this project is to develop a web application that consumes multiple external APIs, processes JSON data asynchronously, stores information locally using localStorage, and provides a responsive interface built with Bootstrap 5.

2.2. Specific Objectives:

Implement the consumption of four different external APIs (JSONPlaceholder, DummyJSON, Dog CEO, and Cat Facts) to obtain and display varied data including users, products, images, and fun facts within a single cohesive application.

Develop a dynamic and responsive user interface using Bootstrap components (tables, cards, modals, and forms) that allows users to view, add, and manage data without requiring page reloads.

Apply asynchronous programming patterns (async/await, try...catch, and fetch) to handle API requests and responses, ensuring proper error management and providing clear user feedback when operations fail.

Implement client-side data persistence using localStorage for manually added users, and compare this approach with other storage mechanisms (sessionStorage and cookies) to understand their appropriate use cases.

4. Development

3.1. Application Structure

The application is organized as a single-page application (SPA) using Bootstrap tabs (nav-tabs). Each tab corresponds to a functional section:

Profile: displays the profile picture (random dog image) and a button to reload it.

Users: table with users from the API and local users, form to add new local users, and buttons to reload or clear data.

Products: catalog in the form of Bootstrap cards with products from DummyJSON.

Cat Facts: button to fetch a fun cat fact and display it in a card.

Research: informative section with comparative tables on Vanilla JS vs jQuery and web storage.

The index.html file contains the complete HTML structure, while app.js houses all the JavaScript logic. Custom styles are defined in style.css



Figure 1 shows the application homepage with all navigation tabs

3.2. Automatic Profile Picture Loading (Dog CEO API)

Functionality: When the page loads, the `loadProfilePicture()` function is executed, consuming the API `https://dog.ceo/api/breeds/image/random` and setting the `src` attribute of the profile image with the result.

Event Used: The `DOMContentLoaded` event was chosen instead of `load` because it fires as soon as the HTML has been parsed, without waiting for external resources like images to load. This allows the profile picture to be requested and displayed immediately, improving the user experience.

```
document.addEventListener("DOMContentLoaded", () => {
  loadProfilePicture();
  loadLocalUsersFromStorage();
  loadUsers();
  loadProducts();
  // ... other initializations
});
```

Code 1 DOMContentLoaded event listener

```
async function loadProfilePicture() {
  const img = document.getElementById("profile-picture");
  setStatus("profile-status", "");
  try {
    const response = await fetch("https://dog.ceo/api/breeds/image/random");
    if (!response.ok) throw new Error("Invalid response from Dog API");
    const data = await response.json();
    img.src = data.message;
  } catch (error) {
    console.error("Error loading profile image:", error);
    setStatus("profile-status", "Could not load profile image. Try again.",
"error");
  }
}
```

Code 2 loadProfilePicture() function

Justification: The image loads automatically without the user pressing any button, meeting the activity requirement. Additionally, a button "Load another image" is provided that invokes the same function.



Figure 2 shows the Profile tab with the loaded image.

3.3. Loading and Displaying Users from JSONPlaceholder

The `loadUsers()` function is asynchronous and uses `fetch` to obtain data from `https://jsonplaceholder.typicode.com/users`. Once the response is received, it is converted to JSON and stored in the global variable `apiUsers`. Subsequently, `renderUsersTable()` is called to render the table.

```
async function loadUsers() {
  setStatus("users-status", "Loading users...");
  try {
    const response = await
fetch("https://jsonplaceholder.typicode.com/users");
    if (!response.ok) throw new Error("Invalid response from
JSONPlaceholder");
    apiUsers = await response.json();
    renderUsersTable();
    setStatus("users-status", `Users loaded: ${apiUsers.length} (API) +
${localUsers.length} (local)`);
  } catch (error) {
    console.error("Error loading users:", error);
    setStatus("users-status", "Could not load users from the API.", "error");
  }
}
```

Code 3 `loadUsers()` function

The table includes the columns: Name, Email, Phone, City, Company, and Origin (with a "API" badge for users from the API). Local users are displayed with a "Local" badge

```

function renderUsersTable() {
  const tbody = document.getElementById("users-table-body");
  tbody.innerHTML = "";

  // Users from API
  apiUsers.forEach((user) => {
    const row = document.createElement("tr");
    row.innerHTML = `
      <td>${user.name}</td>
      <td>${user.email}</td>
      <td>${user.phone}</td>
      <td>${user.address ? user.address.city : "-"}</td>
      <td>${user.company ? user.company.name : "-"}</td>
      <td><span class="badge bg-primary">API</span></td>
    `;
    tbody.appendChild(row);
  });

  // Local users
  localUsers.forEach((user) => {
    const row = document.createElement("tr");
    row.innerHTML = `
      <td>${user.name}</td>
      <td>${user.email}</td>
      <td>${user.phone || "-"}</td>
      <td>${user.city || "-"}</td>
      <td>${user.company || "-"}</td>
      <td><span class="badge bg-success">Local</span></td>
    `;
    tbody.appendChild(row);
  });
}

```

Code 4 renderUsersTable() function

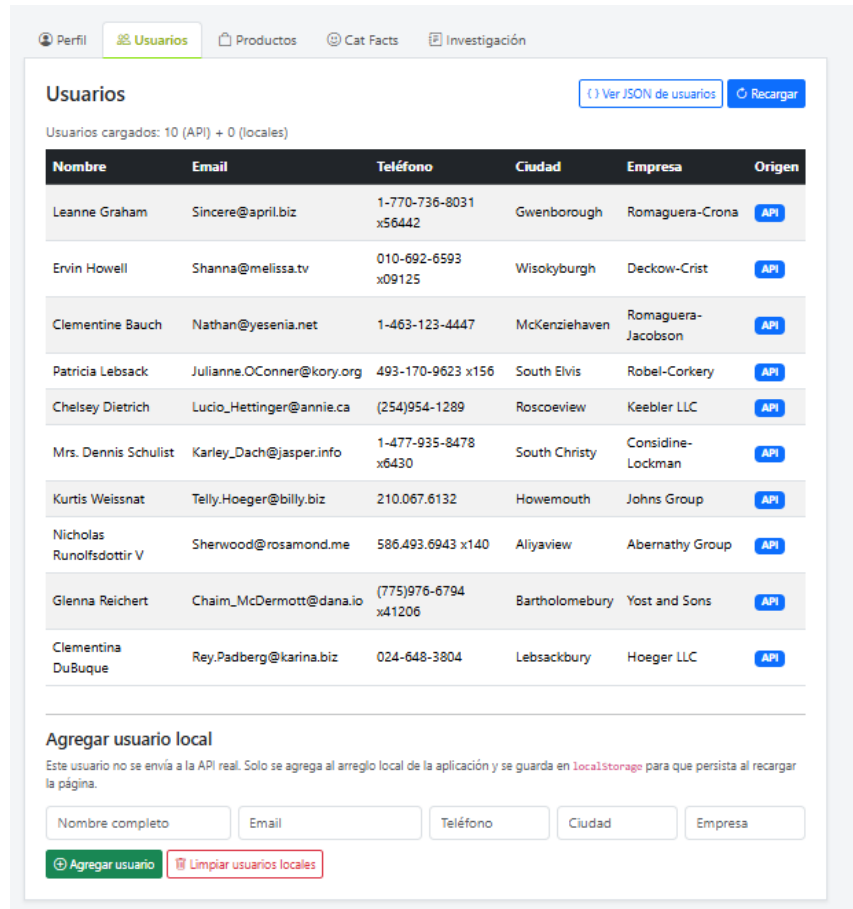


Figure 3 shows the table with API and local users.

3.4. Adding Local Users and Persistence with localStorage

A form has been implemented within the Users tab allowing the user to enter name, email, phone, city, and company. Upon form submission, `handleAddUserForm(event)` executes.

```
function loadLocalUsersFromStorage() {
  try {
    const stored = localStorage.getItem(LOCAL_USERS_KEY);
    localUsers = stored ? JSON.parse(stored) : [];
  } catch (error) {
    console.error("Error reading localStorage:", error);
    localUsers = [];
  }
}
```

Code 5 `loadLocalUsersFromStorage()` function

```
function saveLocalUsersToStorage() {
  try {
    localStorage.setItem(LOCAL_USERS_KEY, JSON.stringify(localUsers));
  } catch (error) {
    console.error("Error saving to localStorage:", error);
  }
}
```

Code 6 saveLocalUsersToStorage() function

```
function handleAddUserForm(event) {
  event.preventDefault();

  const name = document.getElementById("new-user-name").value.trim();
  const email = document.getElementById("new-user-email").value.trim();
  const phone = document.getElementById("new-user-phone").value.trim();
  const city = document.getElementById("new-user-city").value.trim();
  const company = document.getElementById("new-user-company").value.trim();

  if (!name || !email) return;

  const newUser = { name, email, phone, city, company };
  localUsers.push(newUser);

  saveLocalUsersToStorage();
  renderUsersTable();
  setStatus("users-status", `Users loaded: ${apiUsers.length} (API) + ${localUsers.length} (local)`);

  document.getElementById("add-user-form").reset();
}
```

Code 7 handleAddUserForm() function

```
function handleClearLocalUsers() {
  localUsers = [];
  saveLocalUsersToStorage();
  renderUsersTable();
  setStatus("users-status", `Local users deleted. Users loaded: ${apiUsers.length} (API) + 0 (local)`);
}
```

Code 8 handleClearLocalUsers() function

Agregar usuario local

Este usuario no se envía a la API real. Solo se agrega al arreglo local de la aplicación y se guarda en **localStorage** para que persista al recargar la página.

Nombre completo Email Teléfono Ciudad Empresa

+ Agregar usuario Limpiar usuarios locales

Figure 4 shows the add user form with completed fields.

3.5. Storage Comparison: localStorage vs sessionStorage vs Cookies

The Research tab includes a comparative table summarizing the key differences:

Feature	localStorage	sessionStorage	Cookies
Data Duration	Persists until manually deleted	Deleted when the tab/browser is closed	Configurable (can expire in minutes, days, or never)
Capacity	~5-10 MB	~5-10 MB	~4 KB
Sent to Server	No	No	Yes, in every HTTP request
Typical Use Case	Preferences, data that must persist between sessions	Temporary data for a single session	Authentication, session identifiers

Justification for use in this practice: localStorage was chosen to store manually added users because this data needs to persist even after closing and reopening the browser. It is not necessary for the server to have access to this data, so cookies are not required. Nor is it desirable for the data to disappear when the tab is closed, making sessionStorage unsuitable.

Mini Store

Prácticas: APIs, JSON, Async/Await y localStorage

Perfil

Usuarios

Productos

Cat Facts

Investigación

Investigación complementaria

Vanilla JavaScript vs jQuery

Tarea	Vanilla JavaScript	jQuery
Seleccionar un elemento	<code>document.querySelector('#id')</code>	<code>\$('#id')</code>
Escuchar un evento	<code>elemento.addEventListener('click', fn)</code>	<code>\$('#id').on('click', fn)</code>
Solicitar datos a una API	<code>fetch(url).then(...)</code> o <code>await fetch(url)</code>	<code>\$.ajax({url, success: fn})</code>

En este proyecto se utilizó **Vanilla JavaScript** porque el navegador moderno ya incluye `fetch`, `querySelector` y `addEventListener` de forma nativa, con soporte completo para `async/await`. Esto elimina la necesidad de cargar una librería adicional (jQuery) solo para tareas que el propio lenguaje ya resuelve de forma eficiente y con mejor rendimiento.

localStorage vs sessionStorage vs cookies

Característica	localStorage	sessionStorage	Cookies
Duración de los datos	Permanece hasta que se borre manualmente	Se borra al cerrar la pestaña/navegador	Configurable (puede expirar en minutos, días o nunca)
Capacidad aproximada	~5-10 MB	~5-10 MB	~4 KB
¿Se envía al servidor?	No	No	Sí en cada petición HTTP
Caso de uso típico	Preferencias, datos que deben persistir entre sesiones	Datos temporales de una sola sesión (ej. un formulario en progreso)	Autenticación, identificadores de sesión que el servidor necesita leer

En esta práctica se usó **localStorage** para guardar los usuarios agregados manualmente, porque se necesitaba que esos datos **persistieran incluso después de recargar la página**, y no era necesario que el servidor los leyera (a diferencia de una cookie), ni que desaparecieran al cerrar la pestaña (a diferencia de sessionStorage).

Figure 5 shows the Research tab with comparative tables

3.6. Loading and Displaying Products from DummyJSON

The `loadProducts()` function consumes `https://dummyjson.com/products?limit=12` to fetch a list of products.

```
async function loadProducts() {
  const container = document.getElementById("products-container");
  setStatus("products-status", "Loading products...");
  try {
    const response = await fetch("https://dummyjson.com/products?limit=12");
    if (!response.ok) throw new Error("Invalid response from DummyJSON");
    const data = await response.json();
    productsData = data.products;
    renderProducts();
    setStatus("products-status", `Products loaded: ${productsData.length}`);
  } catch (error) {
    console.error("Error loading products:", error);
    container.innerHTML = "";
    setStatus("products-status", "Could not load products from the API.",
"error");
  }
}
```

Code 9 loadProducts() function

Each product is displayed as a Bootstrap card containing: image, title, category (as a badge), brief description, and price with currency formatting.

```
function renderProducts() {
  const container = document.getElementById("products-container");
  container.innerHTML = "";

  productsData.forEach((product) => {
    const col = document.createElement("div");
    col.className = "col-md-4 col-lg-3";
    col.innerHTML = `
      <div class="card product-card h-100 shadow-sm">
        
        <div class="card-body d-flex flex-column">
          <h6 class="card-title">${product.title}</h6>
          <span class="badge bg-secondary badge-category mb-2 align-self-
start">${product.category}</span>
          <p class="card-text small text-muted flex-grow-
1">${product.description}</p>
          <p class="fw-bold text-success mb-0">${product.price}</p>
        </div>
      </div>
    `;
    container.appendChild(col);
  });
}
```

Code 10 renderProducts() function

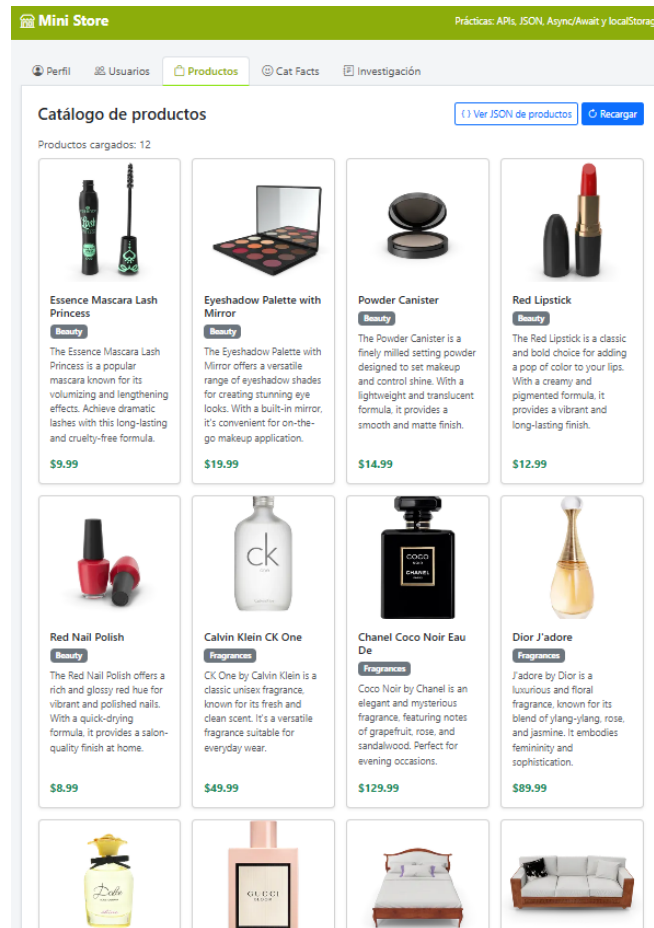


Figure 6 shows the product catalog.

3.7. Basic Store Catalog Simulation

The Products section is designed to resemble an online store catalog. Each card includes an image, title, category, description, and price. A shopping cart is not implemented, but the display is clear and attractive thanks to Bootstrap. Products are loaded when the application starts and can be reloaded using the "Reload" button.

3.8. Displaying Cat Facts

In the "Cat Facts" tab, the user can press the "Show fun fact" button. This executes `loadCatFact()`, which consumes `https://catfact.ninja/fact`. While waiting for the response, a Bootstrap spinner is displayed.

```
async function loadCatFact() {
  const box = document.getElementById("cat-fact-box");
  box.innerHTML = `<div class="spinner-border spinner-inline text-warning"
  role="status"></div> Loading fun fact...`;
  try {
```

```

const response = await fetch("https://catfact.ninja/fact");
if (!response.ok) throw new Error("Invalid response from Cat Facts");
const data = await response.json();
box.innerHTML = `</i> ${data.fact}`;
} catch (error) {
  console.error("Error loading cat fact:", error);
  box.innerHTML = `Could not get the fun fact.
Try again.</span>`;
}
}

```

Code 11 loadCatFact() function

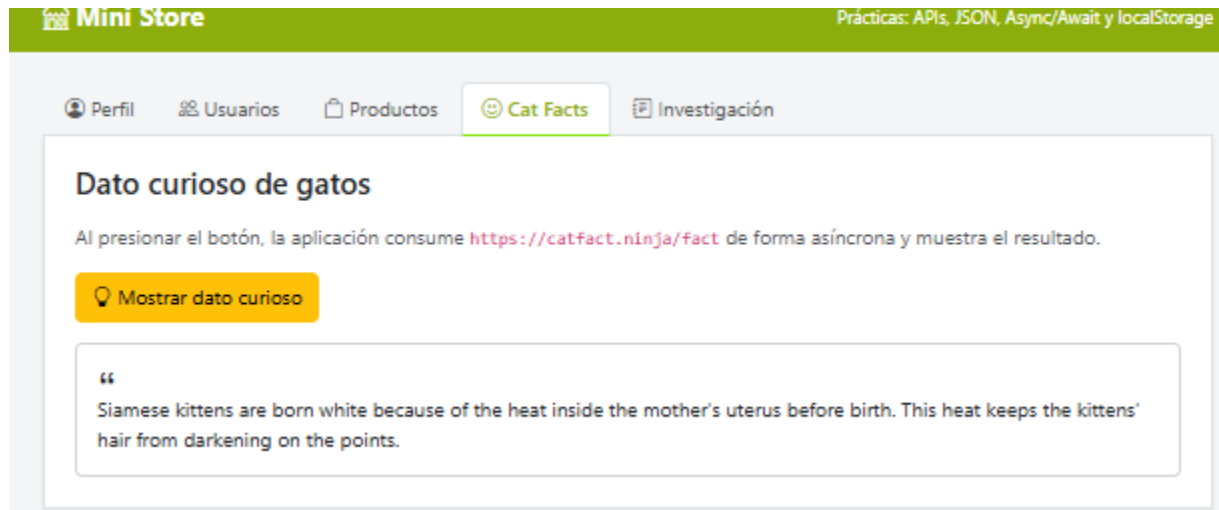


Figure 7 shows a cat fact displayed in the interface

3.9. Displaying JSON in a Modal with JSON.stringify

Two buttons have been added in the Users and Products tabs: "View Users JSON" and "View Products JSON". When clicked, a modal opens displaying the JSON content of the corresponding data.

```

function showJsonModal(title, data) {
  document.getElementById("json-modal-title").textContent = title;
  document.getElementById("json-modal-body").textContent =
    JSON.stringify(data, null, 2);
}

```

Code 12 showJsonModal() function

Explanation: JSON.stringify(data, null, 2) converts a JavaScript value into a JSON string:

data: the object/array to convert

null: second parameter (replacer) - no properties are filtered

2: third parameter (space) - adds 2 spaces for indentation, making it readable

JSON de usuarios (API + locales) X

```
{
  "apiUsers": [
    {
      "id": 1,
      "name": "Leanne Graham",
      "username": "Bret",
      "email": "Sincere@april.biz",
      "address": {
        "street": "Kulas Light",
        "suite": "Apt. 556",
        "city": "Gwenborough",
        "zipcode": "92998-3874",
        "geo": {
          "lat": "-37.3159",
          "lng": "81.1496"
        }
      },
      "phone": "1-770-736-8031 x56442",
      "website": "hildegard.org",
      "company": {
        "name": "Romaguera-Crona",
        "catchPhrase": "Multi-layered client-server neural-net",
        "bs": "harness real-time e-markets"
      }
    },
    {
      "id": 2,
      "name": "Ervin Howell",
      "username": "Antonette",
      "email": "Shanna@melissa.tv",
      "address": {
        "street": "Victor Plains",
        "suite": "Suite 879",
        "city": "Wisokyburgh",
        "zipcode": "98566-7771",
        "geo": {
          "lat": "-43.9509",
          "lng": "-34.4618"
        }
      }
    }
  ]
}
```

Figure 8 JSON Modal showing the users data

JSON de productos X

```
{
  {
    "id": 1,
    "title": "Essence Mascara Lash Princess",
    "description": "The Essence Mascara Lash Princess is a popular mascara known for its volumizing and long-lasting formula.",
    "category": "beauty",
    "price": 9.99,
    "discountPercentage": 10.48,
    "rating": 2.56,
    "stock": 99,
    "tags": [
      "beauty",
      "mascara"
    ],
    "brand": "Essence",
    "sku": "BEA-ESS-ESS-001",
    "weight": 4,
    "dimensions": {
      "width": 15.14,
      "height": 13.08,
      "depth": 22.99
    },
    "warrantyInformation": "1 week warranty",
    "shippingInformation": "Ships in 3-5 business days",
    "availabilityStatus": "In Stock",
    "reviews": [
      {
        "rating": 3,
        "comment": "Would not recommend!",
        "date": "2025-04-30T09:41:02.053Z",
        "reviewerName": "Eleanor Collins",
        "reviewerEmail": "eleanor.collins@x.dummyjson.com"
      },
      {
        "rating": 4,
        "comment": "Very satisfied!",
        "date": "2025-04-30T09:41:02.053Z",
        "reviewerName": "Lucas Gordon"
      }
    ]
  }
}
```

Figure 9 JSON Modal showing the products data

3.10. Using async/await and the Asynchronous Flow

All functions that consume APIs are declared as `async` and use `await` to wait for the resolution of promises returned by `fetch`. The typical flow is:

- The user performs an action (page load, button click).
- An asynchronous function is invoked.
- Inside the function, `await fetch(url)` is used to make the HTTP request.
- Execution pauses until the response is available.
- The response is converted to JSON with `await response.json()`.
- The data is processed, and the DOM is updated.

```
async function loadUsers() {
  setStatus("users-status", "Loading users...");
  try {
    // Step 1: Make the HTTP request
    const response = await
fetch("https://jsonplaceholder.typicode.com/users");

    // Step 2: Check if the response is valid
    if (!response.ok) throw new Error("Invalid response");

    // Step 3: Convert response to JSON
    const data = await response.json();

    // Step 4: Store the data
    apiUsers = data;

    // Step 5: Update the DOM
    renderUsersTable();

    // Step 6: Update status message
    setStatus("users-status", `Users loaded: ${apiUsers.length}`);
  } catch (error) {
    // Step 7: Handle errors
    setStatus("users-status", "Could not load users.", "error");
  }
}
```

Code 13 Complete async/await flow with error handling

This approach avoids "callback hell" and makes the code easier to read.

3.11. Error Handling with try...catch

Every function that consumes an API is wrapped in a try...catch block. If an error occurs, the exception is caught, and an error message is displayed in the interface using the setStatus function.

```
try {
  const response = await fetch("https://dog.ceo/api/breeds/image/random");
  if (!response.ok) throw new Error("Invalid response");
  const data = await response.json();
  img.src = data.message;
} catch (error) {
  console.error("Error:", error);
  setStatus("profile-status", "Could not load the image. Please try again.",
"error");
}
```

Code 14 Error handling pattern

This ensures that the user is informed of failures, improving the application's robustness.

3.12. Comparison: Vanilla JavaScript vs jQuery

The Research tab includes a comparative table showing how common tasks are performed in Vanilla JS and jQuery:

Task	Vanilla JavaScript	jQuery
Selecting an element	document.querySelector('#id')	\$('#id')
Listening to an event	elemento.addEventListener('click', fn)	\$('#id').on('click', fn)
Requesting data from an API	fetch(url).then(...) or await fetch(url)	\$.ajax({url, success: fn})

Conclusion: Vanilla JavaScript was chosen because modern browsers natively support fetch, querySelector, addEventListener, and async/await, eliminating the need for an external library like jQuery. This reduces the application's weight and improves performance

Vanilla JavaScript vs jQuery

Tarea	Vanilla JavaScript	jQuery
Seleccionar un elemento	<code>document.querySelector('#id')</code>	<code>\$('#id')</code>
Escuchar un evento	<code>elemento.addEventListener('click', fn)</code>	<code>\$('#id').on('click', fn)</code>
Solicitar datos a una API	<code>fetch(url).then(...) ○ await fetch(url)</code>	<code>\$.ajax({url, success: fn})</code>

Figure 10 shows the Research tab with comparative tables

5. Conclusions

The use of `fetch`, `async`, and `await` enables the creation of dynamic applications that consume multiple external APIs without requiring page reloads.

`localStorage` is a suitable tool for persisting user-generated data in the browser, eliminating the need for a backend server to store local users.

Error handling with `try...catch` is essential in any application that depends on external services, as these services may fail for reasons beyond the developer's control.

Modern Vanilla JavaScript natively resolves most tasks that previously required jQuery, such as DOM selection, event handling, and HTTP requests, making it a lightweight and efficient choice.

6. References

Bootstrap Team. (2024, February 20). *Bootstrap* Version 5.3.3 CSS and JS framework. <https://getbootstrap.com/docs/5.3/getting-started/introduction/>.

jQuery Foundation. (2026, January 17). *jQuery* Version 4.0.0 JavaScript library. <https://jquery.com/>.

CSS-Tricks. (2017, August 2). *you might not need jQuery*. <https://css-tricks.com/now-ever-might-not-need-jquery/>

Osmani, A. (2020, February 1). *Async/await and promises explained*. freeCodeCamp. <https://www.freecodecamp.org/news/async-await-and-promises/>